

NIGHT WATCH

TECHNISCHE DOKUMENTATION

Escape. Complete Quests. Survive.

Onur Gündüz & Fabian Bechter

Klasse 1AHIF · HTL Rankweil · 22. Juni 2026

Python 3.12+ · pygame 2.6.1
NumPy · Pillow · ~4.400 Zeilen

— 00

Inhaltsverzeichnis



01	Projektbeschreibung	03
02	Spielmechaniken	04
03	Die fünf Aufgaben	05
04	Levelaufbau & Spielwelt	06
05	Monster-KI & Wegfindung	07
06	Technische Architektur	08
07	Module & Klassen	09
08	KI-Verwendung, Änderungen & Fazit	10

01

Projektbeschreibung

Nightwatch ist ein zweidimensionales Top-Down-Horrorspiel, entwickelt als Endabgabe für den POS-Unterricht (Programmieren und Softwareengineering) an der HTL Rankweil. Das Spiel ist vollständig in Python 3.12+ geschrieben und verwendet pygame 2.6.1. Der Spieler erwacht in einem dunklen, mehrstöckigen Haus und muss fünf Aufgaben erfüllen, fünf Schlüssel sammeln und durch den Eingangsbereich fliehen – während ein Monster das Haus patrouilliert. Das Spiel umfasst **~4.400 Zeilen Python** in 17 Dateien.

TECHNISCHER STACK

KOMPONENTE	TECHNOLOGIE	VERWENDUNGSZWECK
Sprache	Python 3.12+	Gesamte Spiellogik
Game-Framework	pygame 2.6.1	2D-Grafik, Sound, Eingabe, Spielschleife
Level-Grafiken	pygame.draw	Alle Level-Grafiken wie Wände, Böden usw.
Film-Grain	NumPy	8 vorberechnete Rauschtexturen
Sound	NumPy + wave	Alle Sounds synthetisch generiert
Menü-Hintergründe	ChatGPT Image	KI-generierte Pixel-Art PNGs (menu_bg, pause_bg)
Kollisionserkennung	pygame.Rect	collidepoint, axis-separierte Bewegung

SPIELZUSTÄNDE (GAMESTATE ENUM)

MENU Hauptmenü mit 3 Buttons	PLAYING Aktives Spielen	PAUSED Pause-Menü	CAUGHT Spieler gefangen
WIN Entkommen + Zeit	NAME_INPUT Highscore-Eingabe	HIGHSCORE Top-10 Liste (JSON)	STEUERUNG Steuerungs-Screen
AUFLÖSUNG 1280×720 intern SCALED Vollbild	WELTKARTE 2400×1100 px 6 Räume + Gänge	PERFORMANCE 120 FPS Ziel Grain vorberechnet	UMFANG ~4.400 Zeilen 17 .py-Dateien

02

Spielmechaniken

STEUERUNG

TASTE	AKTION
W/A/S/D · ↑↓↔	Spieler bewegen
Maus	Taschenlampe ausrichten (360°)
F	Taschenlampe ein/ausschalten
SPACE	Dash (betäubt Monster, 60s Cooldown)
SHIFT	Schleichen (leiser, langsamer)
E	Interagieren / Tür öffnen
P · ESC	Pause-Menü öffnen
F11	Vollbild umschalten

SPIELER-WERTE

Startposition: (340, 250) Weltkoord.
Speed normal: 3.0 px/Frame
Speed schleichend: 1.6 px/Frame
Teppich-Malus: $\times 0.85$ beim Laufen
Kollisionsradius: 14 px
Dash: 9.0 px/F · 18 Frames · 7200F CD
Betäubung: 210 Frames wenn Monster $<130\text{px}$

TASCHENLAMPE & AKKU

Kegel: 280 px lang · 38° Öffnungswinkel
Akku verbraucht: 0.015%/Frame (an)
Akku lädt NICHT auf wenn aus
Bei leer: Lampe schaltet auto aus
Lampe aus → Monster sieht nur 90px
Flackern: alle 300-900 Frames (3-8 F)

BODENTYPEN & LÄRM

TYP	RAUM	HÖRWEITE	LÄRMPEGEL
Parkett	R1, R2, Keller	240 px	1.0 laufen · 0.25 schleichen
Teppich	R3, R4	80 px	0.38 laufen · 0.08 schleichen
Fliesen	Eingang	200 px	1.0 laufen · 0.25 schleichen

BELEUCHTUNGSSYSTEM

1. Dunkle Maske (SRCALPHA) über gesamtem Bildschirm
2. Lichtkegel-Polygon als "Loch" in Maske (30 Segmente)
3. Kerzenlichthöfe (weiche Kreise)
4. Film-Grain (8 NumPy-Frames rotierend)
5. Vignette
6. Gelegentliches Flackern

RENDERING-PIPELINE (9 SCHRITTE PRO FRAME)

Level → Items → Spieler → Monster → Lichtsystem → HUD → Jumpscare → Film-Grain+Vignette → Scale auf Screen

— 03

Die fünf Aufgaben

Fünf Aufgaben sind im Haus verteilt – jede abgeschlossene Aufgabe spawnt einen Schlüssel an ihrer Position. Erst wenn alle 5 Aufgaben erledigt **und** alle 5 Schlüssel eingesammelt sind, öffnet sich die Exit-Tür im Eingangsbereich.

01 CANDLETASK — KERZEN ANZÜNDEN

R1 Wohnzimmer • Parkett

5 Kerzen im Wohnzimmer, jede einzeln mit [E] anzünden (Radius 40 px). Kerzenpositionen: (160,150) · (220,120) · (300,145) · (400,120) · (450,155). Bereits angezündete Kerzen erzeugen einen Lichtschein. Aufgabe gilt als erledigt wenn alle 5 Kerzen brennen.

02 SWITCHTASK — SCHALTER UMLEGEN

R2 Küche • Parkett

3 Lichtschalter in der Küche, alle müssen mit [E] umgelegt werden. Schalter haben visuelle An/Aus-Zustände. Die Reihenfolge spielt keine Rolle – alle 3 müssen aktiv sein.

03 BOXTASK — KISTE TRAGEN

R3 Schlafzimmer • Teppich

Eine Kiste muss vom Startpunkt zur grünen Zielmarkierung getragen werden. Spieler innerhalb 40 px von der Kiste + [E] halten → Kiste folgt dem Spieler. Loslassen von [E] legt die Kiste ab. Aufgabe abgeschlossen wenn Kiste auf Zielposition.

04 MEMORYTASK — MEMORY-PUZZLE

R4 Büro • Teppich

4×4-Tafel mit 16 Memory-Karten (8 Paare). Spieler steht vor der Tafel und drückt [E] zum Interagieren. Zwei Karten aufdecken: stimmen sie überein → bleiben offen; stimmen sie nicht überein → werden wieder verdeckt. Aufgabe erledigt wenn alle 8 Paare gefunden.

05 CODETASK — ZAHLENKOMBINATION

R5 Keller • Parkett

Ein Zahlen-Tresor mit der Kombination **2 · 4 · 1**. Spieler tippt die drei Ziffern (Hinweiszettel liegt im Haus). Falscher Code → Reset. Richtiger Code → Aufgabe erledigt.

SCHLÜSSEL-SYSTEM

Jede abgeschlossene Aufgabe spawnt einen goldenen Schlüssel (mit Bob-Animation) an ihrer Position. Der Spieler muss den Schlüssel aufsammeln. HUD zeigt 5 Kreise oben links – gold wenn gesammelt, grau wenn nicht. Exit öffnet sich nur wenn alle 5 Schlüssel gesammelt sind.

— 04

Levelaufbau & Spielwelt

WELTKOORDINATEN

2400 × 1100 Pixel

BILDSCHIRM INTERN

1280 × 720 Pixel

UMRECHNUNG

 $sx = (wx - cam_x) \times zoom$

— 6 HAUPTRÄUME

#	NAME	POSITION & GRÖSSE	BODEN	AUFGABE
R1	Wohnzimmer	(100,80) 480×340	Parkett	Kerzen anzünden
R2	Küche	(680,80) 480×340	Parkett	Schalter umlegen
R3	Schlafzimmer	(1260,80) 480×340	Teppich	Kiste tragen
R4	Büro	(1840,80) 480×340	Teppich	Memory-Puzzle
R5	Keller	(100,700) 480×340	Parkett	Zahlenkombination (2,4,1)
ENT	Eingang	(680,700) 1640×340	Fliesen	Fluchtweg / Exit-Tür

— 6 VERBINDUNGSGÄNGE

Gang_12: R1↔R2 (580,196) 100×108
Gang_23: R2↔R3 (1160,196) 100×108
Gang_34: R3↔R4 (1740,196) 100×108
Gang_1R5: R1↔Keller (290,420) 100×280
Gang_4EN: R4↔Eingang (2030,420) 100×280
Gang_5EN: Keller↔Eingang (580,800) 100×100

— EXIT-TÜR & ZOOMCAMERA

Exit: Rect(2260, 760, 60, 260)
Interaktion: [E] innerhalb 65px
Bedingung: 5/5 Aufgaben + 5/5 Schlüssel

Zoom = $\min(1280/W, 720/H) \times 0.62$
Min: 0.65 · Max: 1.80 · Lerp: 4.5%/F
ENT-Sonderfall: immer Zoom=1.30

— 05

Monster-KI & Wegfindung

PATROL

Vordefinierte Routen. Langsam.
Zufällig Route 1 oder 2 beim
Start.

ALERT

Geräusch gehört. Geht zur
letzten bekannten Spielerpos.
Sucht 300 Frames.

HUNT

Spieler gesehen. BFS-Verfolgung.
Herzschlag ↑. Jumpscare
ausgelöst.

— MONSTER-WERTE (SCHWIERIGKEITSGRAD: EINFACH — EINZIGER GRAD)

PARAMETER	PATROL	ALERT	HUNT
Speed	0.6 px/F	0.9 px/F	1.5 px/F
Sichtweite (Lampe an)	200 px		
Sichtweite (Lampe aus)	90 px		
Hörweite Parkett	132 px		
Startposition	(1500, 250)		Fangdist: 30px

— NAVIGATIONSGRAPH — 13 KNOTEN · 14 KANTEN

```

Oben:  0(340,250)--1(630,250)--2(920,250)--3(1210,250)--4(1500,250)--5(1790,250)--6(2080,250)
        |                                                                                                     |
Mitte: 7(340,550)                                                                                             13(2080,550)
        |                                                                                                     |
Unten: 8(340,850)--9(630,850)--10(950,850)--11(1500,850)--12(2080,850)-----
  
```

— BFS-ALGORITHMUS (UTILS.PY)

```

def bfs_path(start, ziel):
    queue = deque([[start]])
    visited = {start}
    while queue:
        pfad = queue.popleft()
        if pfad[-1] == ziel: return pfad
        for n in get_nachbarn(pfad[-1]):
            if n not in visited:
                visited.add(n)
                queue.append(pfad+[n])
    return []
  
```

— PATROUILLE-ROUTEN

```

Route 1:
0→1→2→3→4→5
→6→13→12→11
→10→9→8→7→0

Route 2:
3→4→5→6→13
→12→5→4→3
  
```

Technische Architektur

PROJEKTSTRUKTUR

```
nightwatch-abgabe/
├─ src/
│  ├─ main.py          # ~900 Z · GameState, ZoomCamera, Menüs
│  ├─ assets/
│  │  ├─ menu_bg.png   # KI-generiert (ChatGPT Image)
│  │  ├─ pause_bg.png  # KI-generiert (ChatGPT Image)
│  │  ├─ monster_sheet.png # prozedural via Pillow
│  │  └─ player_sheet.png # prozedural via Pillow
│  └─ game/
│     └─ config.py      # ~58 Z · JSON-Konfiguration
│
│  ├─ highscore.py      # ~112 Z · Top-10 JSON
│  ├─ hud.py            # ~216 Z · Akku, Schlüssel, HUD
│  ├─ items.py          # ~691 Z · 5 Aufgaben + Schlüssel
│  ├─ jumpscare.py      # ~137 Z · 3 Effekttypen
│  ├─ level.py          # ~142 Z · Räume, Böden, Exit
│  ├─ lighting.py       # ~188 Z · Maske, Kegel, Grain
│  ├─ monster.py        # ~320 Z · KI-Zustandsmaschine
│  ├─ player.py         # ~198 Z · Bewegung, Dash, Lampe
│  ├─ sounds.py         # ~555 Z · Synthetische Sounds
│  ├─ sprites.py        # ~271 Z · Sprite-Klassen
│  └─ utils.py          # ~115 Z · Vektoren + BFS
│
└─ game_variables/
   └─ game_variables.py # ~180 Z · Zentrale Konstanten (GV)
```

TECHNISCHE BESONDERHEITEN

MAUS-REMAPPING

```
rmx = mx * 1280
// screen_w
```

GV-PATTERN

```
GV = GameVariables()
GV.SCREEN_W
```

PYTHON 3.14

```
pygame.font.init() explizit in GV.init()
```


Module & Klassen

main.py

~900 Z

GameState Enum · ZoomCamera (Lerp-Zoom) ·
FullscreenManager · Menü-Buttons (Hover) ·
Caught/Win/Name-Screen · Hauptspielschleife 120 FPS

items.py

~691 Z

Key (Bob-Anim.) · Task Basis · CandleTask · SwitchTask ·
BoxTask · MemoryTask · CodeTask · ItemManager

sounds.py

~555 Z

SoundManager · Alle Sounds synthetisch via
NumPy+wave · step_hard/soft · torch_click · dash · alert
· jumpscare · heartbeat (lauter bei HUNT)

sprites.py

~271 Z

MonsterSprite (6 Frames × 3 Zustände) · PlayerSprite (4
Lauf + 1 Idle) · Fallback: pygame.draw wenn kein Sheet

monster.py

~320 Z

MonsterState Enum · KI-Zustandsmaschine · BFS-
Navigation · Sicht- und Geräuscherkennung · Stun-
System · apply_difficulty()

hud.py

~216 Z

draw_battery() · draw_keys() · draw_task_list() ·
draw_pulse() · draw_dash_hud() · draw_hint_bar() ·
draw_hud()

lighting.py

~188 Z

SRCALPHA-Dunkelmaske · Lichtkegel 30 Segmente ·
Kerzenlichthof · 8 Film-Grain-Frames (NumPy) · Flackern
· Vignette

utils.py

~115 Z

vec_len · vec_norm · dist · angle_between · lerp · clamp
· bfs_path() · next_waypoint() · nearest_node() ·
_build_adj()

player.py

~198 Z

Bewegung mit Wandkollision (axis-sep.) · Dash-System ·
Taschenlampenrichtung per Maus · Lärmpegel-
Berechnung · Exit-Interaktion

jumpscare.py

~137 Z

3 Typen: HUNT · GEFANGEN · RAUM_R3 · Weißer Blitz
→ rotes Overlay → Vignette · Screen-Shake max 6px ·
200F Cooldown

— 08

KI-Verwendung, Änderungen & Fazit

— KI-VERWENDUNG — 7 MARKIERUNGEN (CLAUDE OPUS 4.8)

#	MODUL	FUNKTION / VERWENDUNGSZWECK
01	main.py	ZoomCamera — Lerp-Zoom, Raum-Verfolgung
02	monster.py	_bewege() — Vektornormalisierung, axis-sep. Kollision, BFS-Navigation
03	lighting.py	_filmkorn_erstellen() — 8 NumPy-Grain-Surfaces mit zufälligem RGBA
04	lighting.py	_lichtkegel_zeichnen() — Bogen-Polygon 30 Segmente, SRCALPHA-"Loch"
05	utils.py	bfs_path() — BFS-Wegfindung auf NAV_NODES/NAV_EDGES Graph

```
# KI CODE ANFANG — Claude Opus 4.8 · # Prompt: "...".  
[Code] · # KI CODE ENDE
```

— WESENTLICHE ÄNDERUNGEN

SCHWIERIGKEITSAUSWAHL ENTFERNT

Spiel startet immer mit "Einfach" — kein Auswahl-Screen mehr

MENÜS NEU GESTALTET

KI-generierte Pixel-Art PNGs als Hintergründe. Hover-Buttons.

JUMPSCARES VEREINFACHT

Von 12 Effektypen (~700 Z) auf 3 (~137 Z). Kein Auto-Timer.

HORROR-DEKORE ENTFERNT

Blutlachen, Wandschriften, Ketten und Schattengestalten entfernt.

DOCSTRINGS VEREINFACHT

""Euklidische Distanz"" → # Abstand zwischen zwei Punkten

FAZIT

Nightwatch ist das umfangreichste Projekt, das wir bisher realisiert haben. Wir sind stolz darauf, dass das Spiel vollständig spielbar ist, eine echte Horror-Atmosphäre erzeugt und ~4.400 Zeilen sauberen Python-Code umfasst. KI wurde für die algorithmisch anspruchsvollsten Stellen genutzt und stets nachvollzogen und markiert.

QUELLEN

- [1] pygame 2.6.1 — [pygame.org/docs/](https://www.pygame.org/docs/)
- [2] NumPy — numpy.org/doc/
- [3] Pillow — pillow.readthedocs.io/
- [4] Python 3 — docs.python.org/3/
- [5] BFS — Wikipedia: Breadth-first search
- [6] Claude Opus 4.8 — anthropic.com

"Escape. Complete
Quests. Survive."

— NIGHTWATCH, 2026